

Entrega Final

Roteamento Dinâmico de Consultas SQL

Baseado em Metadados de Tabelas Apache Iceberg

Aristides Henrique Gonçalves da Cruz

arihenriquede@hotmai.com

Sistemas de Informação

ICEI PUCMinas São Gabriel

Novembro de 2025



PUC Minas

Trabalho: TRABALHO DE CONCLUSÃO DE CURSO
Curso: SISTEMAS DE INFORMAÇÃO
Instituição: PUC MINAS - UNIDADE SÃO GABRIEL

Estrutura da Apresentação

1 Objetivos e Justificativa

- Objetivo Geral
- Objetivos Específicos
- Justificativa

2 Metodologia

- Arquitetura (Diagramas C4)
- Algoritmo de Decisão

3 Expectativas e Resultados

4 Conclusão



Objetivo Geral

Desenvolver o *DyraSQL*, um mecanismo de decisão que utiliza metadados de tabelas *Apache Iceberg* através do comando *EXPLAIN (TYPE IO)* do *Trino* para definir automaticamente o *cluster* mais adequado para execução de consultas SQL.

Problema

Escolha estática de *clusters Trino* (leve versus robusto) sem considerar características dos dados e complexidade das consultas.

Objetivos Específicos

- 1 Mapear e caracterizar os metadados disponíveis em tabelas *Apache Iceberg*, identificando indicadores de volume de dados, complexidade da query e custo esperado de execução.
- 2 Estabelecer regras de decisão para roteamento de consultas, classificando-as em *clusters* leve, intermediário e robusto.
- 3 Projetar e desenvolver um protótipo funcional do *DyraSQL* em ambiente local, utilizando o *Trino*, Docker Compose, MinIO e PostgreSQL.
- 4 Realizar experimentos controlados em cenários distintos de carga de trabalho, verificando a viabilidade técnica e coletando métricas de tempo de execução, consumo de recursos e custo associado.

Justificativa

Necessidade

- Crescimento do volume de dados em *data lakes*
- Necessidade de equilibrar custos e desempenho na execução de consultas SQL
- Escolha estática de *clusters Trino* resulta em provisionamento excessivo ou subdimensionamento

Impacto Esperado

Melhor custo-benefício e utilização eficiente de recursos através de roteamento baseado em análise prévia de metadados.

Revisão Bibliográfica - Fundamentos

Fundamentos Teóricos

- **Apache Iceberg:** Formato de tabela que oferece metadados hierárquicos para gestão eficiente de dados em *data lakes*
- **Trino:** Engine de consultas distribuído que suporta *EXPLAIN (TYPE IO)* para análise prévia de custos
- **Metadados para melhoria:** Informações sobre volume de dados, partições e filtros podem ser utilizadas para estimativa de custo de execução

PUC Minas

Aproveitamento

O *DyraSQL* utiliza o *EXPLAIN (TYPE IO)* do *Trino* para extrair metadados precisos sobre volume efetivo de dados antes da execução.



Revisão Bibliográfica - Trabalhos Relacionados

Trabalhos Principais

- **Xue et al. (2024)**: Sistema adaptativo para execução de consultas em *lakehouse* em larga escala
- **Strausz et al. (2025)**: Otimizador baseado em modelo de custo aprendido para *workloads* SQL *multi-engine*
- **Weintraub et al. (2024)**: Otimização de consultas em *data lakes* baseada em planos de cobertura balanceados

PUC Minas

Diferenciação

O *DyraSQL* propõe roteamento entre *clusters Trino* de capacidades distintas (leve, intermediário e robusto) baseado em análise prévia de metadados do *Apache Iceberg*.



Metodologia

1 Extração de Metadados

- Utilização do *EXPLAIN (TYPE IO)* do *Trino*
- Extração de *outputSizeInBytes*, *outputRowCount*, *cpuCost*

2 Algoritmo de Decisão

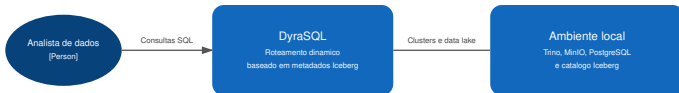
- Cálculo de *Score* baseado em três fatores: volume, complexidade e histórico
- Direcionamento para leve ($\text{Score} < 0,3$), intermediário ($0,3-0,7$) ou robusto ($> 0,7$)

3 Implementação

- *Trino Gateway Proxy* para interceptação de consultas
- *DyraSQL Core* para análise e decisão de roteamento
- Integração com PostgreSQL para histórico e *cache*

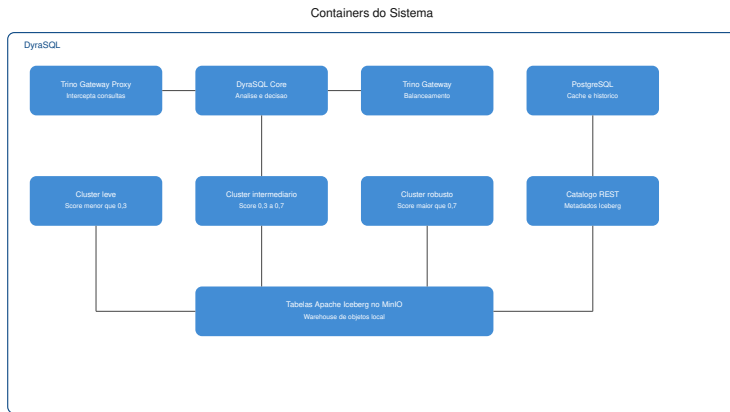
Metodologia - Contexto do Sistema

Contexto do Sistema



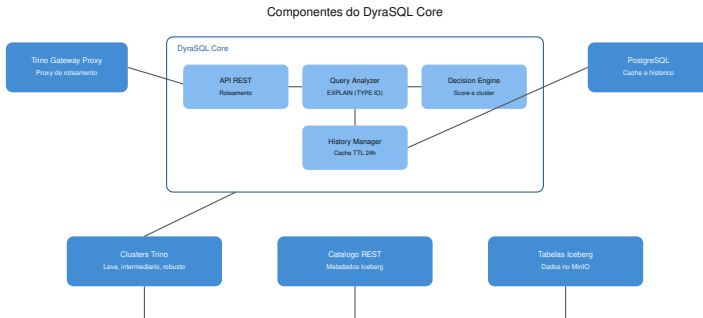
Fonte: Elaborado pelo autor.

Metodologia - Containers do Sistema



Componentes: Trino Gateway Proxy, Trino Gateway, DyraSQL Core, Clusters de Execução

Metodologia - Componentes Internos



Fluxo: Interceptação → Análise → Decisão → Execução → Feedback

Metodologia - Algoritmo de Decisão

Fórmula de Pontuação

$$S = w_1 \times f_v + w_2 \times f_c + w_3 \times f_h \quad (1)$$

- f_v : Fator de volume (50%)
- f_c : Fator de complexidade (30%)
- f_h : Fator histórico (20%)

PUC Minas

Decisão de Roteamento

- Score $< 0,3$: *cluster* leve
- Score $0,3 - 0,7$: *cluster* intermediário
- Score $> 0,7$: *cluster* robusto

Metodologia - Configurações dos Ambientes

cluster leve (Score < 0,3)

- **Memória:** 2 GB de heap
- **Memória máxima por query:** 1 GB
- **Tempo máximo de execução:** 5 minutos
- **Uso:** Consultas leves

cluster intermediário (Score 0,3 – 0,7)

- **Memória:** 3 GB de heap
- **Memória máxima por query:** 2 GB
- **Tempo máximo de execução:** 15 minutos
- **Uso:** Consultas intermediárias

Metodologia - Configurações dos Ambientes (continuação)

cluster robusto (Score > 0,7)

- **Memória:** 4 GB de heap
- **Memória máxima por query:** 3 GB
- **Tempo máximo de execução:** 30 minutos
- **Uso:** Consultas pesadas

Configuração compartilhada

- Catálogo *Iceberg* REST
- MinIO para armazenamento de objetos
- Acesso unificado às tabelas independentemente do *cluster*

Expectativas

Resultados Esperados

- **Precisão:** Sistema capaz de calcular *scores* precisos e direcionar consultas para os *clusters* apropriados
- **Eficiência:** Tempo de análise prévia inferior a 200ms
- **Consistência:** Consultas similares recebem *scores* próximos e são direcionadas para os mesmos tipos de *clusters*
- **Melhoria de Custo-Benefício:** Redução de custos operacionais através de roteamento inteligente

Resultados

Experimentos Realizados

- Dataset sintético em tabelas *Apache Iceberg* no MinIO
- Consultas variadas: diferentes volumes, complexidades e estruturas
- Métricas coletadas: precisão dos *scores*, adequação das decisões, eficiência do sistema



Principais Resultados

- Extração de metadados eficiente: tempo médio < 200ms
- *Scores* precisos e consistentes para consultas similares
- Diferenciação adequada entre consultas leves, intermediárias e pesadas

Desafios da Pesquisa

Desafios Identificados

- **Dependência de Histórico:** Precisão das decisões depende de histórico suficiente no PostgreSQL
- **Limitações do EXPLAIN:** Consultas muito complexas podem exceder 500ms de análise
- **Qualidade dos Metadados:** Dependência da qualidade e atualização dos metadados do *Apache Iceberg*
- **Consultas Atípicas:** Consultas com características não previstas podem resultar em decisões subótimas

Mitigações

Sistema de *cache* temporal, valores *default* para casos sem histórico e aprendizado contínuo baseado em *feedback*.





Dúvidas?